

Q5.2.1) Your implementation is a simplified version of Uijlings et. al approach . In what ways does their version differ from yours?

Uijlings et al. use a more complete selective search pipeline. It has multiple segmentations/scales and a full hierarchical merging strategy to generate strong class independent proposals. Ours is simpler, we generate proposals from our SS output with extra filtering/top-K, then rely more on ResNet features + SVM (and hard negative mining) to decide balloon vs background.

Q5.2.2) What is the effect of changing the thresholds for positive and negative samples? Why do we need two thresholds instead of just one?

Changing t_p and t_n directly changes how clean and how many training samples we get. A higher t_p makes positives more precise but fewer. A higher t_n makes negatives stricter, a lower t_n adds more negatives but can include “almost balloon” regions that confuse the classifier. We need two thresholds because proposals in the middle IoU range are ambiguous. they are neither clear positives nor clear negatives. Using two thresholds lets us label only confident positives and confident negatives and ignore uncertain samples, which usually improves training stability.

Q5.2.3) The balloon dataset is quite small and does not provide many training samples. Can you think of ways to increase the amount of training data?

Use data augmentation (jitter/scale/shift, flips, color/brightness), run selective search with more settings to create more proposals, and add more hard negatives/near-miss samples through hard negative mining.

```

(.venv) uw64eqyr@cip4a0:~/Desktop/CV/Ex-5/code$ python option2.py --data_root "
/uw64eqyr/Desktop/CV/Ex-5/data/balloon_dataset" \
> --outdir cache_5_2_submit --cmd eval --split test \
> --scales "50,100,200,400,800" --sigma 0.8 --min_size 10 \
> --model cache_5_2_submit/svm_resnet18.joblib
MABO (proposals) on test: 0.7198
loading annotations into memory...
Done (t=0.00s)
creating index...
index created!
Eval category_id used for detections: 1
Total detections written: 5504
Loading and preparing results...
DONE (t=0.10s)
creating index...
index created!
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=0.03s).
Accumulating evaluation results...
DONE (t=0.00s).
Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.032
Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.080
Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.010
Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = -1.00
Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.059
Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.038
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 1 ] = 0.052
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.184
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.380
Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = -1.00
Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.321
Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.567
(.venv) uw64eqyr@cip4a0:~/Desktop/CV/Ex-5/code$ python option2.py --data_root "

```

Figure 1: ...

On the test split, our selective search proposals achieve a MABO of 0.7198, which indicates that for most ground-truth balloons there exists at least one proposal with a fairly good overlap. However, the full detection pipeline (proposals + ResNet18 features + linear SVM) achieves a relatively low COCO mAP of 0.032 (AP@0.50 = 0.080, AP@0.75 = 0.010). This gap shows that proposal coverage is not the main bottleneck; instead, many detections are either not tight enough to reach high IoU (hurting AP@0.75) or false positives are ranked too highly (hurting precision). The “small” category returns -1.00, meaning there are effectively no small objects evaluated in this split or not enough matches, while medium and large objects reach AP values of 0.059 and 0.038, respectively.

Commands Pipeline to train and evaluate the code

1) proposals

```

python Individualsolution.py --data_root "D : 4project5dataset" --outdir cache52submit --
--cmdpropose --splittrain --force --scales "50, 100, 200, 400, 800" --sigma 0.8 --minsize 10
python Individualsolution.py --data_root "D : 4project5dataset" --outdir cache52submit --cmdpropose --
--splitvalid --force --scales "50, 100, 200, 400, 800" --sigma 0.8 --minsize 10

```

```
python Individualsolution.py--dataroot" D : 4project5dataset"'--outdircache52ssubmit--cmdpropose--
--splittest'--scales" 50, 100, 200, 400, 800"--sigma0.8--minsize10
```

2) Train

```
python Individualsolution.py--dataroot" D : 4project5dataset"'--outdircache52ssubmit--cmdtrain--
--force'--scales" 50, 100, 200, 400, 800"--sigma0.8--minsize10'--tp0.50--tn0.30--hnmrounds2--
--hnmkeepperimage200
```

3) Evaluation

```
python Individualsolution.py--dataroot" D : 4project5dataset"'--outdircache52ssubmit--cmdeval--
--splittest'--scales" 50, 100, 200, 400, 800"--sigma0.8--minsize10'--model" cache52ssubmitresnet18.joblib
```

4) infer + visualize

```
python Individualsolution.py--dataroot" /home/cip/ai2025/uw64eqyr/Desktop/CV/Ex-5/data/baloondataset
--outdircache52ssubmit--cmdinferone--image" /home/cip/ai2025/uw64eqyr/Desktop/CV/Ex-
5/data/baloondataset/test/485227412e335662bb5bpg.rf.c3b02e09097409dfeb31d85ce92175b2.jpg"--
--scales" 50, 100, 200, 400, 800"--sigma0.8--minsize10--modelcache52ssubmit/svmresnet18.joblib
--scorethr1.0--nmsiou0.020--topdet10
```